

# Telegram-ассистент по расписанию общественного транспорта

GTFS + MCP - итоговый отчёт

Город: Каунас, Литва · Фид: mdb-1041 (Mobility Database) · Дата отчёта: 25.09.2026

---

## 1. Архитектура: путь от сообщения до ответа

Пассажир → Telegram → bot.py → agent.py (логика) → MCP-сервер → SQLite (данные GTFS).

### 1.1 Данные

GTFS Schedule Каунаса (Kauno viešasis transportas) скачан с Mobility Database и проверен официальным GTFS Validator: 0 errors, 294 warnings, 25 infos - все замечания косметические (регистр названий, незаполненные необязательные поля). Скрипт gtfs\_loader.py разбирает zip-архив и загружает 10 таблиц в SQLite (data/gtfs.db) с индексами под ключевые запросы.

### 1.2 MCP-сервер (server.py + gtfs\_queries.py)

Пять инструментов: find\_stop, get\_next\_departures, get\_routes\_through\_stop, get\_fare, plan\_trip.

Вся логика расчётов - обычный Python и SQL, без участия языковой модели: календарь работы (calendar + calendar\_dates), расстояние между точками (формула Хаверсина), маршрут с одной пересадкой - через пересечение множеств «куда можно доехать напрямую от А» и «откуда можно доехать напрямую до В». Сервер работает самостоятельно, без модели и без Telegram - проверялся через MCP Inspector на каждом этапе разработки.

### 1.3 Агент (agent.py, логика переиспользована в bot.py)

Google Gemini API получает список инструментов сервера и по тексту вопроса решает, что вызвать. Реализован цикл многошаговых вызовов (до 8 шагов на вопрос) - модель может вызвать несколько инструментов подряд (например, найти обе конечные точки маршрута, затем построить сам маршрут), прежде чем ответить словами. История диалога хранится в памяти и передаётся модели целиком при каждом вызове - так работает контекст на один шаг («а следующий?»).

### 1.4 Telegram-бот (bot.py)

Тонкая обёртка: принимает текст и геопозицию, передаёт вопрос в ту же агентную логику. У каждого чата - своя независимая история диалога. Один MCP-сервер поднимается один раз при старте бота, а не на каждое сообщение.

## 1.5 Проверочный контур (evaluation/)

questions.json - 50 вопросов, поровну по пяти обязательным типам, с эталонами, полученными независимо от бота - напрямую через SQL-скрипты check.py и check\_trip.py, до того как агент на них тестировался. run\_eval.py прогоняет весь набор через агента одной командой и считает метрики.

## 1.6 Почему так, а не иначе

- а) SQLite, а не pandas - для запроса «маршрут с пересадкой» нужны JOIN'ы по нескольким таблицам с фильтрацией по календарю; на фиде такого размера (69 маршрутов) накладные расходы SQLite не критичны, а запросы читаются проще, чем эквивалентная логика на DataFrame.
- б) Каунас, а не другой город - 0 ошибок в валидаторе (в отличие от рассмотренного Хельсинки, где 2 ошибки и ~1 ГБ данных - риск для недельного срока), 69 маршрутов (выше порога в 50), есть данные о ценах (fare\_rules) для честной демонстрации частичного покрытия.
- в) Gemini, а не другая модель - доступ через бесплатный API, поддержка function calling; выбор конкретной версии модели потребовал нескольких итераций (раздел 3.4).
- г) Разделение сервер / агент / бот - сервер работает и тестируется независимо от остальных частей (требование ТЗ); agent.py и bot.py используют идентичную логику вызова инструментов, чтобы не дублировать и не рассинхронизировать поведение.

## 2. Метрики проверки: определения

Из 50 вопросов проверочного набора каждый получает один из вердиктов:

- 1. correct - ответ агента содержит все ожидаемые факты.
- 2. partial - найдена часть, но не все ожидаемые факты.
- 3. incorrect - ожидаемые факты не найдены вообще.
- 4. error - агент не дал ответа (сбой API, исчерпан лимит запросов, превышен лимит шагов).
- 5. manual\_review - для trip\_planning автоматический вердикт не ставится: эталон - текстовое описание маршрута, надёжно сверить строкой было бы нечестно; ответы сохранены для ручного прочтения.

## Результаты последнего полного прогона

| Категория                | correct | partial | incorrect | error | manual review |
|--------------------------|---------|---------|-----------|-------|---------------|
| stop_search (12)         | 9       | 1       | 0         | 2     | -             |
| next_departures (12)     | 8       | 1       | 0         | 3     | -             |
| routes_through_stop (10) | 5       | 0       | 0         | 5     | -             |
| trip_planning (6)        | -       | -       | -         | 1     | 5             |
| unsupported (10)         | 7       | 0       | 3         | 0     | -             |
| Итого (50)               | 29      | 2       | 3         | 11    | 5             |

## Что эта таблица не видит

1. Все 11 «error» - инфраструктурные, не логические. Это сбои лимита запросов бесплатного тарифа Gemini (429/503), а не ошибки в понимании вопроса или в данных. Там, где агент успевал ответить, ответ был по существу верным.
2. 1 из 3 «incorrect» в unsupported - ложный, ошибка грейдера, а не бота. Вопрос «Кто сейчас водитель автобуса №1?» - бот честно ответил «нет доступа к информации», но список ключевых слов в `run_eval.py` не включал фразу «нет доступа» (только «недоступ» слитно). После обнаружения список расширен.
3. Автоматические вердикты - поиск ключевых слов/чисел в тексте, не проверка смысла. Теоретически возможны ложные «correct», если число упомянуто не в том контексте.
4. Каждый вопрос прогонялся с чистой историей диалога - таблица не проверяет контекст на 1 шаг («а следующий?»). Он проверялся отдельно, вручную.
5. Вопросы уже содержат явную дату и время («после 10:00 25.09.2026») - баг, описанный в разделе 3.3 (тихая подмена времени), срабатывал только когда время передавалось без даты - то есть в разговорных уточнениях, а не в структурированных вопросах набора. Поэтому данные этого прогона, по нашей оценке, не искажены этим багом.

### 3. Разбор ошибок: что пошло не так и что с этим сделано

#### 3.1 Модель не выдумывает факты - базовое требование выполняется

Во всех наблюдавшихся случаях (и в evaluation, и в ручном тестировании через Telegram) агент не подставлял придуманные времена, маршруты или цены. Категория unsupported стабильно получает честное «не знаю» без попыток угадать.

#### 3.2 Агент перебирал варианты вместо уточняющего вопроса

При нескольких найденных остановках с одинаковым названием (например, «Ateities pl. A» и «B») первая версия агента не спрашивала пользователя, а перебирала варианты сама вызовами инструментов - тратила весь лимит шагов, не успевая ответить. Причина: правило «уточнять, а не гадать» было только в описании самого инструмента `find_stop`, а не в общей инструкции агенту. Исправлено явным правилом в `system_instruction`. После правки агент действительно переспрашивает - подтверждено и в CLI-тесте, и в Telegram.

#### 3.3 Серьёзный баг: время без даты тихо игнорировалось

Обнаружен не автотестом, а живым использованием через Telegram: вопрос «а после него?» (продолжение диалога про рейсы, последний из которых был в 17:18) неожиданно вернул рейсы в 19:19–19:20 - разрыв в два часа. Сверка с независимым эталоном (`check.ru`) показала, что между 17:18 и 19:19 реально существуют 5 рейсов, которые бот пропустил.

Причина - в `server.py`: обработка параметров (`date`, `time`) не покрывала случай «передано только `time`, без `date`» - он проваливался в ветку по умолчанию и подставлял настоящее текущее время сервера (Алматы, UTC+5) вместо запрошенного времени по Каунасу. Баг был продублирован в двух функциях (`get_next_departures` и `plan_trip`), потому что код был скопирован, а не переиспользован. Исправлено: логика вынесена в общую функцию `_parse_target_datetime()`, используется в обоих инструментах.

**Важный вывод для методологии:** ни MCP Inspector (ручные разовые вызовы), ни автоматический проверочный набор (все вопросы содержат явную дату и время) не могли обнаружить этот баг - он проявлялся только в конкретном разговорном сценарии без явной даты. Обнаружен исключительно благодаря ручному тестированию живого диалога через Telegram-бота.

#### 3.4 Смена модели Gemini

Первая выбранная модель (`gemini-2.5-flash`) оказалась недоступна для нового аккаунта (404). Следующая (`gemini-3.8-flash`) заработала, но лимит бесплатного тарифа - 20 запросов в день, что позволяет обработать буквально 5–6 вопросов. Модель `gemini-2.0-flash` оказалась полностью снята с поддержки (404, «no longer available»). Остановились на `gemini-3.1-flash-lite` - работает, лимит заметно выше. Урок: доступность и лимиты бесплатных моделей меняются быстрее ожидаемого; при повторной сборке проекта эту часть стоит перепроверить.

### 3.5 Лимит запросов - не дневной, а в минуту

Часть ошибок 429 при прогоне evaluation изначально считались «провалом агента», хотя на деле были лимитом запросов в минуту, о котором сама ошибка API сообщала точное время ожидания. Добавлен автоматический retry с парсингом предложенной задержки - до 5 попыток на запрос.

## 4. Известные ограничения (честно, не скрываем)

6. `plan_trip` не проверяет рейсы после полуночи (в отличие от `get_next_departures`, где это учтено) - для дневных вопросов не критично, для поздних/ранних поездок с пересадкой возможны неполные результаты.
7. Геопозиция в Telegram-боте не протестирована вручную (тестировщик географически удалён от данных Каунаса) - логика идентична уже проверенной через MCP Inspector, но живого подтверждения через реальное устройство нет.
8. Автоматический грейдер evaluation - эвристика по ключевым словам, не семантическая проверка; несколько ложных срабатываний уже найдено и задокументировано (раздел 2).
9. 11 вопросов из 50 не получили ответа при последнем прогоне исключительно из-за лимитов бесплатного API - не по вине логики агента.

## 5. Что бы сделали дальше

1. Повторный прогон `run_eval.py` после исправления бага с `time/date` и после расширения списка «не знаю»-маркеров - для получения чистых метрик без известных искажений.
2. Ручная проверка всех 6 ответов категории `trip_planning` против эталонов (сейчас помечены `manual_review`).
3. Переход на платный тариф Gemini (копейки при таком объёме) или на альтернативный провайдер с более щедрым бесплатным лимитом (Groq) - чтобы инфраструктурные еггор не занижали видимое качество агента.
4. Добавить обработку рейсов после полуночи в `plan_trip`, аналогично `get_next_departures`.
5. Выборочная ручная проверка части «correct»-вердиктов - чтобы убедиться, что автоматический грейдер не даёт ложных срабатываний в обратную сторону.

## Заключение

Разработан воспроизводимый прототип Telegram-ассистента, в котором факты о расписании вычисляются MCP-сервером на основе GTFS, а языковая модель отвечает за выбор инструментов и формулировку ответа. Все пять обязательных типов вопросов реализованы. Основные оставшиеся ограничения связаны с лимитами внешнего LLM API, автоматической оценкой ответов и поездками после полуночи.